

Spark / Databricks Interview Prep — Part 2 of 4

Modules 8-14 | 30 Questions & Answers per Module | 5 YOE Data Engineer Level

Module 8: Physical Execution

- Q1. [Scenario]** For `SELECT department, COUNT(*) FROM employees GROUP BY department`, why two `HashAggregate` nodes? A: The first `HashAggregate` runs partial (map-side) aggregation before the shuffle, combining rows locally per partition to reduce shuffle volume; the second `HashAggregate` runs final (reduce-side) aggregation after the shuffle, combining the partial results by department into the final count.
- Q2. [Scenario]** `.explain()` shows `SortMergeJoin` where you expected `BroadcastHashJoin`. First checks? A: Verify actual table size vs `spark.sql.autoBroadcastJoinThreshold`, confirm statistics are present/fresh (`ANALYZE TABLE`), check whether the "small" side is actually the result of a complex subquery/aggregation Spark can't size-estimate accurately, and check if AQE would switch it post-shuffle.
- Q3. [Scenario]** Two `Exchange` nodes back-to-back with nothing between them — red flag? A: Usually a red flag — it suggests two separate wide operations that could potentially be consolidated or reordered to avoid a redundant shuffle; worth investigating whether both are truly necessary given the logic.
- Q4. [Scenario]** `HashAggregate` vs `SortAggregate` — when does Spark pick each? A: `HashAggregate` is preferred (in-memory hash table per group) when it fits memory constraints and aggregate functions support it; `SortAggregate` is used as a fallback for aggregate functions/types not supported by hash-based aggregation, or when data is already sorted appropriately, trading memory for guaranteed correctness/scalability via sort-based grouping.
- Q5. [Scenario]** `WholeStageCodegen` wraps multiple operators — what does that mean? A: Those operators are fused into a single generated JVM method executed as one tight loop per row, avoiding the overhead of separate iterator-based calls between each operator — a key Tungsten/codegen performance benefit.
- Q6. [Trap]** Does the physical plan always mirror the logical plan structurally? A: No — physical planning can significantly restructure the plan, e.g., splitting a single logical aggregate into partial + final `HashAggregate` physical nodes around a shuffle.
- Q7. [Trap]** Does `Project` always mean columns are being added? A: No — it also represents column selection/pruning and can be entirely optimized away if it's a no-op relative to the existing schema.
- Q8. [Trap]** Is `FileScan` always the first/only scan operator? A: Not necessarily — multiple `FileScan` leaves can appear if the query joins or unions multiple source tables.
- Q9. [Trap]** Does `.explain()` show the AQE-adjusted runtime plan by default? A: No — the default pre-execution `.explain()` shows the initial plan; the actual AQE-adjusted plan is best viewed via the Spark UI's SQL tab after execution completes.
- Q10. [Coding]** `df.groupBy("dept").count().explain("formatted")` — how to identify partial vs final aggregation? A: Look for two `HashAggregate` nodes in the physical plan tree, separated by an `Exchange`; the one before the `Exchange` is the partial/map-side aggregation, the one after is the final/reduce-side aggregation.
- Q11. [Coding]** `GROUP BY` on a UUID column — does partial aggregation still happen, and is it useful? A: Partial aggregation still occurs, but with near-unique keys almost no rows combine locally, so it adds overhead (extra hash table construction/lookup) without meaningfully reducing shuffle volume — a case where the "combiner" optimization provides little benefit.

Q12. [Output] What does `Exchange hashpartitioning(dept#12, 200)` mean? A: Data is being shuffled into 200 partitions using a hash of the `dept` column to determine the target partition for each row.

Q13. [Output] What does `BroadcastExchange` imply about task distribution? A: That side's complete dataset is being sent (replicated) to every executor rather than being partitioned/shuffled — each task on the large side has full local access to the small side's data for the join.

Q14. [Internals] Why does Spark do partial aggregation before the shuffle? A: To reduce the volume of data that must be shuffled across the network — combining rows locally first (like a MapReduce "combiner") means far less data needs to travel over the wire before final aggregation.

Q15. [Internals] How do statistics influence physical operator choice? A: Row count, table size, and column statistics (from `ANALYZE TABLE`, Delta stats, or file metadata) inform the cost-based optimizer's decisions — e.g., whether a table qualifies for broadcast join, or the estimated join order for multi-table queries.

Q16. [Internals] Why is reading `.explain("formatted")` so valuable? A: It reveals exactly what operators, join strategies, and shuffle boundaries Spark chose, letting you verify assumptions (e.g., confirming a broadcast join actually happened) and pinpoint exactly where a query's cost is coming from, rather than guessing from code alone.

Q17. [Follow-up] How would you confirm which join strategy Spark actually used at runtime vs the initial plan predicted? A: Check the Spark UI's SQL tab after the job completes — it shows the (potentially AQE-adjusted) executed plan annotated with runtime metrics, which reflects the true executed strategy.

Q18. [Follow-up] If stats are stale, what physical-plan mistakes could result? A: Failing to broadcast a table that has since shrunk below the threshold (missing a cheap optimization), or broadcasting a table that has since grown too large (risking driver/executor OOM) — both stemming from outdated size assumptions.

Q19. [Follow-up] Risk of blindly forcing `broadcast()` hints everywhere? A: Driver or executor OOM if the hinted table isn't actually small (or has grown over time), since explicit hints override the safety check normally provided by the auto-broadcast threshold.

Q20. [Scenario] A physical plan shows a `Filter` operator placed after a `HashAggregate` rather than before the scan. Why might this be correct, not a missed optimization? A: The filter likely references an aggregated column (e.g., `HAVING count(*) > 5`), which can only be evaluated after aggregation produces that value — such filters correctly cannot be pushed below the aggregation.

Q21. [Trap] Does a `BroadcastHashJoin` guarantee no shuffle occurs anywhere in the query? A: No — only the joined tables' redistribution is avoided for that specific join; other parts of the query (e.g., a subsequent `groupBy`) can still introduce their own shuffle/`Exchange` nodes.

Q22. [Coding] How would you force Spark to display the physical plan with runtime metrics for a completed query via code (not just the UI)? A: Use `df.explain(mode="formatted")` for the static plan pre-execution; for actual runtime metrics, query the Spark REST API's SQL execution endpoints, or programmatically access `spark.sharedState.statusStore` / `QueryExecutionListener` hooks to capture executed metrics.

Q23. [Scenario] Why might `SortAggregate` appear even when `spark.sql.execution.useObjectHashAggregateExec` and hash aggregation are enabled? A: Certain aggregate functions (e.g., ones with specific ordering requirements, or unsupported data types for hash-based state) force Spark to fall back to sort-based aggregation regardless of configuration, since hash aggregation isn't universally applicable.

Q24. [Internals] What's the difference between "logical" and "physical" statistics used in planning?

A: Logical-level statistics describe the abstract relation (row counts, distinct value estimates from `ANALYZE TABLE`); physical-level statistics can include file-level metadata (Parquet footer stats, file sizes) used more directly during physical planning and scan pruning — both feed into cost estimation but at different granularities.

Q25. [Follow-up] How would you diagnose a case where the correct join strategy was chosen but performance is still poor?

A: Investigate skew within that join strategy (e.g., `SortMergeJoin` skew on a hot key), insufficient shuffle partitions, serialization overhead, or resource contention — join strategy correctness doesn't guarantee absence of other performance issues.

Q26. [Scenario] Would you expect `Exchange` to appear for a `LIMIT` clause on an already-sorted, single-partition `DataFrame`?

A: Not necessarily — if data is already effectively centralized/ordered appropriately, Spark can often satisfy a small `LIMIT` without a full shuffle, though a global limit after operations spread across many partitions may still require some coordination step to gather top-N results correctly.

Q27. [Trap] Does the presence of `WholeStageCodegen (1)`, `(2)`, etc., numbering indicate sequential execution order?

A: The numbers are just identifiers for separate generated code blocks/pipeline segments, not necessarily a strict execution order indicator on their own — refer to the overall plan tree structure to understand actual data flow and dependencies.

Q28. [Coding] How would you check if column pruning actually reduced the number of columns read from a wide table in the physical plan?

A: Inspect the `FileScan`/`Project` node's listed output schema in `.explain()` — it should list only the referenced columns, not the full table schema, confirming pruning occurred.

Q29. [Scenario] Two functionally identical queries — one written in SQL, one in `DataFrame` API — produce different physical plans. Is that expected?

A: It shouldn't be expected for genuinely equivalent logic — since both compile through the same Catalyst pipeline, differences usually indicate the queries aren't actually logically identical (e.g., subtle differences in column references, join conditions, or implicit casts) rather than an API-level discrepancy.

Q30. [Follow-up] What's your general workflow for using `.explain("formatted")` output to debug a

production performance issue? A: Start by scanning for unexpected `Exchange` nodes (extra/avoidable shuffles), verify join strategies match expectations given table sizes, check `FileScan` for pushed filters and pruned columns, look for `WholeStageCodegen` gaps (indicating UDF/interpreted fallback), then correlate plan structure with Spark UI stage/task metrics for the actual runtime bottleneck.

Module 9: Whole-Stage Code Generation

Q1. [Scenario] All-native pipeline shows a single `WholeStageCodegen` block wrapping several operators — what's happening at the JVM level?

A: Spark generates a single Java method that inlines the logic of all fused operators (filter, project, etc.) into one tight per-row loop operating directly on Tungsten's binary row format, compiled to JVM bytecode and JIT-optimized — avoiding the overhead of separate virtual-function calls between operators.

Q2. [Scenario] A single Python UDF in the middle of an otherwise native pipeline — how does it break codegen fusion?

A: UDF execution requires leaving the JVM's generated-code execution path entirely to invoke the Python interpreter (via serialization across the process boundary); this forces the plan to split

into separate segments around the UDF, each with its own (potentially non-codegen) execution path, breaking the single fused loop.

Q3. [Scenario] Native `when()/col()` pipeline vs equivalent Python UDF pipeline show very different codegen behavior — why, and what perf difference? A: The native version stays fully within Catalyst/Tungsten's codegen-eligible execution, compiled into efficient JVM bytecode; the UDF version forces a break-out to Python per row/batch, incurring serialization and interpreter overhead — commonly resulting in an order-of-magnitude (or more) slowdown for the UDF version at scale.

Q4. [Trap] Does whole-stage codegen eliminate shuffles? A: No — it only optimizes CPU-bound, per-row processing within pipelined (narrow) operator chains; shuffle boundaries (`Exchange` nodes) remain separate execution stages unaffected by codegen fusion.

Q5. [Trap] Does every physical operator benefit from whole-stage codegen? A: No — some operators (certain complex joins/sorts, or operators not yet implemented for codegen) fall back to traditional Volcano-style iterator execution rather than being fused into generated code.

Q6. [Trap] Is codegen generating Python bytecode? A: No — it generates and JIT-compiles JVM bytecode; Python is never involved in the codegen path itself (only in UDF execution, which is separate from and disruptive to codegen).

Q7. [Coding] How do you identify which operators got fused under a single `WholeStageCodegen` node in `.explain("formatted")`/Spark UI? A: Operators nested under a `WholeStageCodegen (N)` marker in the physical plan tree (or visually grouped in the Spark UI's SQL tab DAG visualization) are fused together; operators outside that grouping (e.g., after an `Exchange` or a UDF break) execute separately.

Q8. [Coding] Edge case: "Generated method too long" (JVM 64KB method-size limit) — what happens? A: Spark falls back to non-codegen/interpreted execution for that stage/operator segment when the generated code would exceed the JVM's method bytecode size limit — a known edge case for extremely wide/complex expressions.

Q9. [Output] Is `Exchange` part of the same codegen block as surrounding operators? A: No — shuffle boundaries are never fused into codegen blocks; `Exchange` always marks a break between codegen segments (and stages).

Q10. [Internals] Why can native functions outperform Python UDFs specifically because of codegen (not just serialization)? A: Native operations get compiled into a single tight JVM loop with no per-row abstraction/virtual-call overhead, operating directly on Tungsten's binary format; UDFs force interpreted per-row (or per-batch for Pandas UDFs) calls plus, for Python UDFs, cross-process serialization — both codegen loss and serialization cost compound the slowdown.

Q11. [Internals] How does Tungsten's binary row format relate to codegen? A: Generated code operates directly on Tungsten's compact, contiguous binary row representation rather than JVM object graphs, avoiding per-row object allocation/pointer-chasing overhead and improving CPU cache locality within the generated loop.

Q12. [Internals] Typical order-of-magnitude difference between codegen-optimized native pipelines and UDF-broken pipelines at scale? A: Commonly 2x-10x+ (sometimes more for row-heavy Python UDFs), though the exact factor depends on data volume, UDF complexity, and batch vs row-at-a-time execution.

Q13. [Follow-up] If codegen is so much faster, why doesn't every operator support it? A: Generating correct, general-purpose code for all operator variants/edge cases is complex and error-prone; some operators fall back to interpreted execution for correctness/maintainability reasons rather than risking subtle codegen bugs.

Q14. [Follow-up] How would you detect in production that codegen got disabled/bypassed for a critical stage? A: Compare expected vs actual stage CPU time, inspect the physical plan for missing `WholeStageCodegen` wrapping around operators you expect to be fused, check for UDF usage breaking the chain, or look for codegen fallback warnings in driver/executor logs.

Q15. [Follow-up] Does DataFrame/SQL API always benefit from codegen more than RDD API? Why? A: Yes — RDDs operate on generic JVM objects without the schema-awareness Catalyst/Tungsten need to generate optimized, type-specialized code; the structured DataFrame/SQL API provides the schema information codegen relies on.

Q16. [Scenario] Would replacing a chain of 10 `.withColumn()` calls with a single `.select()` containing 10 expressions change codegen behavior? A: Not meaningfully in modern Spark — Catalyst optimizes/normalizes both into an equivalent projection during logical plan optimization before physical planning, so codegen fusion outcomes should be effectively the same either way.

Q17. [Trap] Does enabling `spark.sql.codegen.wholeStage=false` ever make sense in production? A: Rarely — mainly for debugging suspected codegen-related bugs/crashes (a known historical pain point with extremely complex generated code); disabling it in normal production trades away significant performance for debugging convenience, so it should be temporary.

Q18. [Coding] How would you benchmark the actual performance delta between a native expression and a UDF-based equivalent on your own data? A: Run both versions on the same data/cluster, compare total stage duration and per-task CPU time in the Spark UI, and inspect `.explain()` for presence/absence of `WholeStageCodegen` wrapping around the relevant operator to correlate the performance gap with the codegen difference.

Q19. [Scenario] A join operator shows up outside any `WholeStageCodegen` block even though both sides are otherwise fully native. Is that expected? A: Yes, potentially — certain join implementations (e.g., `SortMergeJoin`) are only partially codegen-eligible or interact with codegen differently than simple projections/filters; not every native operator is guaranteed full fusion, this is implementation-dependent.

Q20. [Follow-up] Would switching from `SortMergeJoin` to `BroadcastHashJoin` change codegen fusion behavior for that join? A: Potentially — different join physical operators have different codegen support/characteristics; verifying via `.explain("formatted")` for each specific case is more reliable than assuming based on join type alone.

Q21. [Scenario] Does adding `.cache()` in the middle of a native pipeline affect codegen fusion around it? A: Yes — caching materializes data at that point, which acts similarly to a stage boundary for codegen fusion purposes (the cached read and subsequent operators would be in separate codegen segments from whatever produced the cached data).

Q22. [Trap] Is "vectorized" Parquet reading the same thing as whole-stage codegen? A: No — they're complementary but distinct: vectorized Parquet reading is a columnar batch-read optimization at the I/O/scan layer; whole-stage codegen is a separate execution-engine optimization for per-row processing logic downstream of the scan.

Q23. [Internals] How does codegen interact with `CASE WHEN`/complex expressions performance-wise compared to equivalent UDF logic? A: Complex but native `CASE WHEN` expressions remain codegen-eligible and get compiled into efficient conditional JVM bytecode within the fused loop, while equivalent UDF logic (even if simple) forces a process boundary crossing — native expressions are almost always preferable when expressible.

Q24. [Scenario] Would increasing executor cores help mitigate the performance loss from codegen-breaking UDFs? A: Only marginally/partially — more cores add parallelism but don't address the

fundamental per-row serialization and interpreter overhead of UDF execution; replacing the UDF with native logic (or a Pandas UDF for necessary custom logic) is the more effective fix.

Q25. [Follow-up] Would a Pandas UDF restore codegen fusion around it? A: No — Pandas UDFs still represent a break from the JVM-native codegen path (execution moves to Python, batched via Arrow), though the batching reduces the *frequency* of the JVM↔Python transition compared to row-at-a-time UDFs, improving performance without restoring full codegen fusion.

Q26. [Scenario] A junior engineer asks "should I always avoid UDFs entirely to preserve codegen?" How do you respond? A: Not entirely — prefer native functions whenever functionally equivalent logic exists, since it preserves codegen/Catalyst optimization; reserve UDFs (ideally Pandas UDFs) for genuinely custom logic native functions can't express, understanding the tradeoff you're accepting for that flexibility.

Q27. [Coding] How would you inspect whether a specific stage's high CPU time correlates with codegen-fused execution or interpreted fallback? A: Cross-reference the Spark UI's per-task CPU time for that stage against the `.explain("formatted")` physical plan — if the corresponding operators are wrapped in `WholeStageCodegen`, high CPU time is expected/efficient; if they sit outside any codegen block (e.g., around a UDF), high CPU time may indicate the interpreted-execution overhead you should target for optimization.

Q28. [Trap] Does codegen apply during shuffle read/write serialization itself? A: Not in the same sense — shuffle read/write involves its own serialization mechanisms (potentially Tungsten-optimized for DataFrame/SQL operations), but "codegen" as discussed here specifically refers to per-row processing logic fusion within a stage, not the shuffle transport mechanism itself.

Q29. [Scenario] Would you expect codegen benefits to be more or less pronounced on a CPU-bound vs I/O-bound job? A: More pronounced on CPU-bound jobs — codegen's benefit is reducing per-row CPU/interpretation overhead; for I/O-bound jobs (e.g., dominated by network/disk wait), the relative impact of codegen on overall wall-clock time is smaller since CPU processing isn't the bottleneck.

Q30. [Follow-up] How would you explain the business impact of "avoid UDFs for codegen" to a non-technical stakeholder? A: Frame it as: "using Spark's built-in operations lets the engine auto-generate highly optimized, compiled code for our logic; custom Python code can't get that same optimization, so wherever we can express our logic using standard operations, the job runs meaningfully faster and cheaper for the same result."

Module 10: Tungsten

Q1. [Scenario] RDD of Python objects vs DataFrame — why does the DataFrame version use less memory/CPU from a Tungsten perspective? A: DataFrames leverage Tungsten's schema-aware binary encoding, storing data in a compact, contiguous off-heap-friendly format rather than as boxed JVM/Python objects with per-object overhead and pointer indirection — reducing both memory footprint and GC pressure, and enabling codegen over that binary format.

Q2. [Scenario] Team debates migrating legacy RDD jobs to DataFrame API "just for syntax" — what's the deeper Tungsten argument? A: Beyond syntax, the DataFrame API unlocks Tungsten's memory management (off-heap binary rows, reduced object overhead), Catalyst's plan optimizations, and whole-stage code generation — none of which apply to raw RDD-of-objects code, meaning the migration is a genuine performance upgrade, not just a readability one.

Q3. [Trap] Is Tungsten a separate system from Spark SQL, or an execution engine component? A: An execution engine optimization layer within Spark (primarily activated through the structured Spark SQL/DataFrame/Dataset APIs), not a standalone separate product.

Q4. [Trap] Do Tungsten's benefits apply equally to RDD and DataFrame APIs? A: No — the schema-aware binary format and codegen benefits are mainly realized through the structured DataFrame/SQL API; raw RDD operations on arbitrary JVM objects largely bypass these optimizations.

Q5. [Trap] Does Tungsten eliminate garbage collection entirely? A: No — it significantly reduces GC pressure via off-heap/binary memory management for structured data, but doesn't eliminate GC entirely, since JVM overhead, driver-side objects, and non-Tungsten-managed structures still exist.

Q6. [Coding] How would you empirically demonstrate Tungsten's benefit to a teammate? A: Run an equivalent RDD-of-objects `.map()`-heavy job and a DataFrame-based job on the same data/cluster, then compare task GC time, memory usage, and overall duration in the Spark UI — the DataFrame version should show markedly lower GC time and often faster completion.

Q7. [Output] What Spark UI signal indicates a job not benefiting from Tungsten's memory management? A: High "GC Time" relative to total task duration — a sign of heavy JVM object churn typical of RDD/UDF-heavy code that bypasses Tungsten's binary format.

Q8. [Internals] List Tungsten's core focus areas and explain each in one line. A: Memory management (efficient allocation/off-heap use), binary data representation (compact row format avoiding object overhead), CPU efficiency (fewer wasted cycles per operation), cache efficiency (contiguous memory improving CPU cache hit rates), code generation (compiling logic to optimized JVM bytecode), reduced object overhead (avoiding per-record JVM object allocation).

Q9. [Internals] Why did Spark introduce Tungsten? A: To move beyond the limitations of JVM object overhead and garbage collection pressure inherent in earlier RDD-based execution, pursuing CPU/cache-efficient, off-heap binary processing that closes the performance gap with hand-optimized, lower-level code.

Q10. [Internals] How does Tungsten's binary row format improve CPU cache locality vs JVM object graphs? A: It stores row data compactly and contiguously in memory (similar to a C-style struct layout) rather than as scattered JVM objects connected by pointers, meaning sequential row processing accesses memory in cache-friendly patterns instead of triggering cache misses from pointer-chasing.

Q11. [Internals] How does off-heap memory management under Tungsten relate to

`spark.memory.offHeap.enabled`? A: When enabled, Tungsten can allocate a portion of its managed memory outside the JVM heap entirely, further reducing garbage collector involvement/pause times for that memory region, at the cost of needing to explicitly size and manage that off-heap pool via

`spark.memory.offHeap.size`.

Q12. [Follow-up] Does Tungsten help Python UDF performance? A: No — UDF execution leaves the JVM/Tungsten-optimized execution path entirely to run in a separate Python process; Tungsten's benefits don't extend across that boundary.

Q13. [Follow-up] How does Tungsten relate to Project Catalyst — are they the same project? A: Complementary but distinct: Catalyst optimizes the logical/physical query *plan* (what operations to do and in what order); Tungsten optimizes the low-level *execution* of those operations (memory layout, CPU efficiency, code generation) — together they form Spark SQL's optimization stack.

Q14. [Follow-up] Migration argument to leadership grounded in Tungsten benefits for a legacy RDD pipeline? A: Reduced GC pauses and memory footprint, better CPU cache utilization, eligibility for whole-

stage codegen, and consequently likely significant (often multi-fold) speedups and potential cost savings from reduced cluster resource needs for the same workload.

Q15. [Scenario] A team caches a large RDD of custom case-class objects vs an equivalent cached DataFrame. Which uses less cluster memory, and why? A: The DataFrame version — Tungsten's binary encoding for cached structured data is significantly more memory-compact than the JVM object representation used for cached RDD objects (which include object headers, boxing, and pointer overhead per field).

Q16. [Trap] Does converting an RDD to a DataFrame via `.toDF()` automatically gain full Tungsten benefits retroactively for logic already executed as RDD transformations upstream? A: No — only the operations expressed through the DataFrame/SQL API downstream of the conversion benefit from Tungsten/Catalyst; upstream RDD-based logic that already executed (or logic still expressed as RDD transformations even after conversion, e.g., via `.rdd.map()`) doesn't retroactively gain these optimizations.

Q17. [Coding] How would you identify in the Spark UI whether cached storage is using Tungsten's compact encoding vs generic Java serialization? A: Check the "Storage" tab's reported size for the cached DataFrame/RDD — a DataFrame cached via `MEMORY_ONLY`/`MEMORY_AND_DISK` using Tungsten encoding will typically show a notably smaller memory footprint than an equivalent RDD of objects cached with default Java serialization, for the same underlying data.

Q18. [Scenario] Would explicitly using `MEMORY_ONLY_SER` on an RDD close the memory gap with a Tungsten-backed DataFrame? A: It narrows the gap (serialized storage is more compact than raw Java objects) but doesn't fully close it — Tungsten's binary format is purpose-built for structured/schema-aware data and generally remains more compact and more directly usable (without deserialization) than generic RDD serialization.

Q19. [Internals] Why is "reduced object overhead" listed as a distinct Tungsten benefit from "binary data representation" — aren't they the same thing? A: They're related but distinct facets: binary data representation refers to the compact on-the-wire/in-memory row layout itself; reduced object overhead refers more specifically to avoiding the JVM's per-object bookkeeping (headers, boxing of primitives, reference indirection) that would otherwise be incurred processing each record as a discrete object.

Q20. [Follow-up] If a candidate says "Tungsten is just about using less memory," how would you correct/expand that framing? A: Memory efficiency is one major outcome, but Tungsten's scope also includes CPU efficiency (fewer wasted cycles, better cache locality) and enabling code generation — it's a holistic execution-efficiency initiative, not solely a memory optimization.

Q21. [Scenario] Would Tungsten's benefits be more or less pronounced on a cluster with abundant memory but constrained CPU? A: More pronounced on CPU-constrained clusters — since a meaningful part of Tungsten's value is reducing CPU cycles spent on object overhead/GC and improving cache efficiency, CPU-bound workloads benefit disproportionately compared to purely memory-bound ones.

Q22. [Trap] Does disabling whole-stage codegen also disable Tungsten's binary memory format? A: No — they're separate mechanisms; disabling codegen (`spark.sql.codegen.wholeStage=false`) falls back to interpreted (Volcano-style) execution but can still operate over Tungsten's binary row format, just without the generated-code fusion benefit.

Q23. [Coding] How would you measure the GC time difference attributable to Tungsten specifically, isolating it from other factors? A: Run a controlled A/B comparison (same data, same cluster, same logic) — one path via RDD-of-objects, one via equivalent DataFrame operations — and compare only the "GC Time" Spark UI metric between the two runs, keeping all other variables constant.

Q24. [Scenario] Would a very small dataset (a few MB) show a noticeable Tungsten-driven performance difference between RDD and DataFrame execution? A: Likely minimal/negligible — Tungsten's benefits scale with data volume and per-row processing overhead; for tiny datasets, fixed overheads (task scheduling, JVM startup) dominate far more than the memory/CPU efficiency gains Tungsten provides.

Q25. [Follow-up] Is it accurate to say "Tungsten made Spark SQL as fast as hand-written Scala/Java code"? A: That was explicitly Tungsten's stated aspiration/goal (closing the gap toward hand-optimized code performance) — it's a reasonable characterization of the intent, though "as fast as" is an idealized framing; real-world performance still depends on workload specifics.

Q26. [Scenario] Does Tungsten's off-heap memory management interact with

`spark.executor.memoryOverhead` **sizing considerations?** A: Yes conceptually — if off-heap memory is enabled and sized via `spark.memory.offHeap.size`, that allocation is separate from the JVM heap but still counts toward the executor container's total memory footprint, which must be accounted for (alongside `memoryOverhead`) when sizing executors to avoid container-level OOM kills by the cluster manager.

Q27. [Trap] Is "off-heap" memory the same thing as "storage memory" in the unified memory model?

A: No — "off-heap vs on-heap" is about *where* the memory physically lives relative to the JVM heap; "storage vs execution" (within the unified memory model) is about *what the memory is used for* (caching vs computation) — these are orthogonal dimensions, and both on-heap and off-heap memory can, in principle, be allocated to either storage or execution purposes depending on configuration.

Q28. [Coding] How would you demonstrate to a skeptical teammate that a specific slow job would benefit from moving off raw RDD object manipulation to DataFrame operations, using only Spark UI evidence (no code changes yet)? A: Point to elevated GC time and larger-than-expected memory usage per task in the Executors/Stages tabs for the RDD-based stages, and contrast with typical GC/memory profiles of comparable DataFrame-based stages elsewhere in the pipeline (or from a small proof-of-concept rewrite) to make the case concrete rather than theoretical.

Q29. [Scenario] Would rewriting a UDF-heavy DataFrame pipeline to use only native functions primarily be "a Tungsten win" or "a Catalyst win," and does the distinction matter in an interview

answer? A: It's really both, and articulating the distinction shows depth: Catalyst gains come from renewed eligibility for pushdown/pruning/reordering around the removed UDF; Tungsten gains come from the resulting operations now running over the binary format with codegen rather than breaking out to Python — a strong interview answer names both mechanisms rather than treating "it got faster" as a single undifferentiated improvement.

Q30. [Follow-up] How would you summarize the Catalyst vs Tungsten distinction in one sentence for a whiteboard interview? A: "Catalyst decides the smartest *plan* for what to do; Tungsten makes sure that plan *runs* as efficiently as possible at the CPU/memory level."

Module 11: Serialization

Q1. [Scenario] Python UDF on 1B rows is far slower than native functions — trace the serialization

path. A: For each row (or micro-batch), data must be serialized from the JVM's internal representation, sent across a process boundary (historically via a socket/pipe using Python's pickle format, or Arrow for batched paths) to a separate Python worker process, deserialized into Python objects, processed by the UDF, re-serialized, and sent back to the JVM to be deserialized again — this round trip repeated at row/batch granularity across a billion rows is the dominant cost.

Q2. [Scenario] Switching `spark.serializer` to Kryo for RDD-heavy jobs improves shuffle time — why mechanistically? A: Kryo produces more compact serialized representations and serializes/deserializes faster than the default Java serializer, directly reducing the bytes written/read during shuffle and the CPU time spent on (de)serialization, both of which shrink shuffle write/read duration.

Q3. [Scenario] A closure captures a 50MB lookup dictionary from the driver — what's the serialization cost, and per what unit? A: The dictionary must be serialized as part of the task's closure and shipped to executors — potentially per task (depending on task/closure caching behavior) rather than once, multiplying network and serialization overhead significantly; this is precisely the scenario a broadcast variable is designed to fix (serialize/ship once per executor, not per task).

Q4. [Trap] Is Kryo used by default in Spark? A: No — Java serialization is the default; Kryo must be explicitly configured via `spark.serializer`, though DataFrame/SQL operations largely bypass generic serializers anyway via Tungsten's specialized encoders.

Q5. [Trap] Do DataFrame/Dataset operations rely on the same generic serialization as RDDs? A: Mostly no — structured APIs use Tungsten's schema-aware encoders for known types, which are far more efficient than generic reflection-based Java/Kryo object serialization used for arbitrary RDD objects.

Q6. [Trap] Does Kryo improve Python UDF performance? A: No — Kryo is a JVM-side serializer affecting JVM object/shuffle serialization; Python UDF overhead is a separate JVM↔Python boundary cost entirely unaffected by the JVM's internal serializer choice.

Q7. [Coding] How do you register a custom class with Kryo to avoid slower reflection-based fallback? A: Configure `spark.kryo.registrator` with a custom `KryoRegistrator` implementation, or use `conf.registerKryoClasses(Array(classOf[MyClass]))`, explicitly registering classes so Kryo can use optimized, pre-registered serialization instead of falling back to slower generic reflection-based handling for unregistered classes.

Q8. [Coding] Edge case: an unregistered, non-serializable class — what error and where? A: Typically a `NotSerializableException` (or Kryo equivalent), surfacing either during closure cleaning/task serialization on the driver before dispatch, or during task execution/deserialization on the executor — depending on exactly when serialization is attempted.

Q9. [Output] Are serialization time and computation time reported separately in Spark UI? A: Yes — metrics like "Task Deserialization Time" are shown distinctly from actual task execution/computation time, allowing you to isolate serialization overhead specifically.

Q10. [Internals] Compare Java vs Kryo serialization on size, speed, ease of use. A: Java serialization: larger output, slower, works out-of-the-box with no configuration (uses reflection, implements `Serializable`). Kryo: more compact output, faster, requires (or benefits significantly from) explicit class registration for best performance, slightly more setup effort.

Q11. [Internals] Describe the JVM → Python → JVM round trip for a Python UDF in detail. A: The JVM serializes the relevant row/column data (historically per-row via pickling over a socket; increasingly via Arrow-based batched transfer for Pandas UDFs), sends it to a spawned Python worker process, the worker deserializes, executes the Python function, serializes the result, sends it back over the same channel, and the JVM deserializes it back into its internal representation to continue the pipeline.

Q12. [Internals] Why is serialization overhead a first-class distributed-systems concern generally? A: Any time data crosses a process, machine, or language boundary, it must be converted to a transportable format and back — this conversion consumes CPU and adds latency proportional to data volume and per-unit overhead, making serialization efficiency a fundamental lever in any distributed or multi-process system's overall throughput.

Q13. [Follow-up] If DataFrames use Tungsten encoders, why does `spark.serializer=kryo` still matter? A: It still matters for RDD-based code paths, certain internal shuffle structures, and object caching scenarios that fall outside the fully Tungsten-optimized DataFrame execution path — many real pipelines mix RDD and DataFrame code, or use caching/collect patterns where Kryo's efficiency still helps.

Q14. [Follow-up] How would switching a UDF to a Pandas UDF change the serialization story? A: It shifts from per-row pickled serialization to batched, columnar Arrow-based serialization — dramatically fewer round trips (per batch instead of per row) and a more efficient, low-copy columnar transfer format, substantially reducing serialization overhead even though the JVM↔Python boundary itself still exists.

Q15. [Follow-up] Describe a real scenario where closure serialization caused a production issue. A: A representative example: a `map()` closure accidentally captured a driver-side database connection object (non-serializable, and conceptually invalid to use on executors), causing immediate `NotSerializableException` failures at job submission — resolved by moving connection creation inside the executor-side function (e.g., per-partition, via `mapPartitions`) rather than capturing it from the driver's scope.

Q16. [Scenario] Would enabling Kryo serialization change the physical plan Catalyst chooses for a DataFrame query? A: No — the choice of serializer is an execution-engine-level setting largely orthogonal to Catalyst's plan selection (which is based on cost/statistics for join strategy, pushdown, etc.); it affects how data is encoded during shuffle/caching, not which logical/physical operators are chosen.

Q17. [Trap] Does registering a class with Kryo guarantee correctness if that class has custom equals/hashCode logic? A: Registration primarily affects serialization efficiency, not correctness of equals/hashCode semantics; if a class's serialized-then-deserialized round trip doesn't preserve the properties your logic depends on (e.g., custom object identity semantics), that's a separate correctness concern independent of the registration itself.

Q18. [Coding] How would you benchmark Java vs Kryo serialization impact on your own shuffle-heavy RDD job? A: Run the identical job twice, toggling `spark.serializer`, and compare "Shuffle Write Time," "Shuffle Read Time," and total shuffle bytes in the Spark UI, along with overall job duration, holding all other configuration and data constant.

Q19. [Scenario] A closure references `self.some_field` inside a class method used as a UDF/map function — what serialization risk does this introduce beyond referencing a local variable? A: Referencing `self.field` implicitly captures the entire enclosing object `self` (not just that field) for serialization, which can pull in far more state (potentially non-serializable or unexpectedly large) than intended — a classic subtle closure-serialization pitfall, often mitigated by extracting only the needed value into a local variable before the closure, or using a broadcast variable.

Q20. [Follow-up] How would Spark's closure cleaner help or not help in the `self.field` scenario above? A: Spark's closure cleaner attempts to strip genuinely unused references from a closure before serialization, but it can't always perfectly untangle deeply nested object references (especially across language boundaries in PySpark); it's a best-effort mitigation, not a guaranteed fix, so being deliberate about what you capture remains important.

Q21. [Scenario] Would using `spark.serializer=kryo` ever increase job duration compared to Java serialization? A: In rare cases, if class registration is incomplete and Kryo falls back to slower generic/reflection-based serialization paths for unregistered classes with complex structures, the overhead of that fallback could occasionally underperform a straightforward Java serialization path — proper registration is important to realize Kryo's benefits.

Q22. [Trap] Is serialization overhead only relevant for RDD/UDF code, never for pure DataFrame pipelines? A: Not entirely — even pure DataFrame pipelines incur serialization during shuffle (Tungsten-optimized, but not free) and when collecting results to the driver; the *magnitude* of concern is much lower than generic RDD/UDF serialization, but it isn't literally zero.

Q23. [Coding] How would you inspect whether a specific job's slowness correlates with high deserialization time using the Spark UI? A: Check the "Task Deserialization Time" column/metric in the Stages tab's task table — consistently high values relative to total task duration across many tasks point to serialization (not computation) as a significant contributor to that stage's runtime.

Q24. [Scenario] Would broadcasting instead of using a closure variable ever be worse for a very small (a few KB) lookup value? A: For genuinely tiny values, the overhead difference is negligible either way, and a plain closure variable may be simpler; broadcast variables provide their clearest benefit specifically as the shared data grows large enough that repeated per-task serialization becomes meaningfully costly.

Q25. [Internals] What role does `spark.kryoserializer.buffer.max` play, and when would you need to adjust it? A: It caps the maximum size of the Kryo serialization buffer; if you attempt to serialize an object (e.g., a very large broadcast variable or closure) exceeding this limit, serialization fails with a buffer-overflow-related error, requiring you to increase this setting for legitimately large objects.

Q26. [Follow-up] How would you decide whether a serialization bottleneck is best solved by switching serializers, reducing data volume, or restructuring the closure/broadcast usage? A: Diagnose first (via Spark UI deserialization/shuffle metrics) whether the issue is overall data volume (favoring reducing/filtering data or better partitioning), repeated small-object overhead (favoring broadcast variables), or general per-byte serialization inefficiency (favoring a faster serializer like Kryo) — each symptom points to a different fix, and applying the wrong one wastes effort.

Q27. [Scenario] Would a Scala closure capturing a large immutable case class have the same "repeated serialization per task" risk as the Python dictionary example? A: Yes, structurally the same risk applies in Scala/JVM closures too — any sufficiently large captured object shipped as part of a task's closure repeatedly (rather than broadcast once) incurs redundant serialization/network cost, regardless of language; broadcast variables address this JVM-side as well as in PySpark.

Q28. [Trap] Does using a broadcast variable eliminate serialization entirely for that data? A: No — the data is still serialized, just far less often (once per executor rather than once per task), which is the actual efficiency gain; serialization itself as a mechanism isn't eliminated, only its repeated redundant cost is minimized.

Q29. [Coding] How would you verify that a broadcast variable is actually being reused across tasks rather than re-serialized each time, empirically? A: Compare "Task Deserialization Time" and network I/O metrics before and after converting a large closure variable to a broadcast variable for the same job/data — a significant drop in per-task deserialization overhead and total network traffic is strong empirical evidence of the fix taking effect.

Q30. [Follow-up] What's your production checklist for avoiding serialization-related surprises when writing new Spark jobs? A: Avoid capturing large or non-serializable objects (DB connections, SparkSession, large collections) directly in closures; use broadcast variables for meaningfully large shared read-only data; prefer native DataFrame operations over RDD/UDF code where possible to minimize reliance on generic serialization; and register custom classes with Kryo when RDD-based serialization is unavoidable and performance-sensitive.

Module 12: Python UDF / Pandas UDF / Arrow

Q1. [Scenario] Replacing native `when/otherwise` with a Python UDF "for readability" — quantify the regression and root causes. A: Expect a potentially significant (often multi-fold) slowdown at scale; root causes: loss of Catalyst-level optimization eligibility (pushdown/pruning around the UDF), loss of whole-stage codegen fusion, and per-row cross-process serialization overhead moving data to/from the Python interpreter — three compounding costs a native expression avoids entirely.

Q2. [Scenario] Applying an ML model's `.predict()` per row — compare row UDF, Pandas UDF, and native Spark ML pipeline. A: Row-at-a-time Python UDF: simplest to write, worst performance (per-row serialization + interpreter overhead). Pandas UDF: batches rows via Arrow, leverages vectorized numpy/pandas/model inference, much better throughput, moderate implementation effort. Native Spark ML pipeline: best integration with Catalyst/distributed training-serving patterns if the model type is natively supported, but least flexible for arbitrary custom models (e.g., external framework models) not natively integrated.

Q3. [Scenario] Pandas UDF works on small data, OOMs on executors at scale — why? A: Each batch materializes as an in-memory pandas Series/DataFrame on the executor; large batch sizes (`spark.sql.execution.arrow.maxRecordsPerBatch`) or wide/high-cardinality data per batch can still exceed available executor memory — vectorization doesn't eliminate memory bounds, it just changes the unit of processing from row to batch.

Q4. [Scenario] Minimal improvement switching row UDFs to Pandas UDFs — what else might be the bottleneck? A: Arrow not actually enabled (`spark.sql.execution.arrow.pyspark.enabled=false`), a very small `maxRecordsPerBatch` limiting batching benefit, or the UDF's own logic (not the transport mechanism) being the true bottleneck — e.g., an inherently slow algorithm inside the function regardless of how efficiently data reaches it.

Q5. [Trap] Are Pandas UDFs guaranteed faster than Python UDFs in all cases? A: No — for very lightweight per-row logic on small data, vectorization/batching overhead may not pay off compared to simplicity of row UDFs; the advantage grows with data scale and computation complexity per row.

Q6. [Trap] Do Pandas UDFs completely eliminate the JVM↔Python boundary cost? A: No — they reduce the *frequency* of crossings (batched via Arrow) rather than eliminating the boundary itself; the JVM↔Python transition still exists per batch, just amortized over many rows instead of paid per single row.

Q7. [Trap] Does enabling Arrow automatically speed up all Python UDFs? A: No — Arrow mainly benefits `toPandas()`/`createDataFrame()` conversions and Pandas UDFs specifically; plain row-at-a-time `udf()` functions don't automatically gain Arrow's batching benefit just because Arrow is enabled globally.

Q8. [Trap] Is a Scala UDF as slow as a Python UDF? A: No — Scala UDFs execute natively within the JVM with no cross-process serialization boundary, though they still bypass some Catalyst-level optimization (pushdown) and full codegen fusion compared to fully native expressions — meaning Scala UDFs sit performance-wise between native expressions and Python UDFs, generally much closer to native.

Q9. [Coding] Write a Pandas UDF that doubles a numeric column, contrasted with the row-based equivalent. A:



```
python
```

```
# Row-based UDF
from pyspark.sql.functions import udf
from pyspark.sql.types import DoubleType
double_udf = udf(lambda x: x * 2, DoubleType())
df.withColumn("y", double_udf(df.x))
```

```
# Pandas UDF (vectorized)
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf(DoubleType())
def double_pandas(x: pd.Series) -> pd.Series:
    return x * 2

df.withColumn("y", double_pandas(df.x))
```

The Pandas UDF operates on a whole batch (`pd.Series`) at once using vectorized pandas arithmetic, versus the row UDF's Python interpreter call per individual value.

Q10. [Coding] Edge case: a Pandas UDF returns a Series of different length than the input batch. A:

This is an error — Series-to-Series (scalar) Pandas UDFs must return output of exactly the same length as the input batch; mismatched lengths raise a runtime exception.

Q11. [Coding] Edge case: null handling differences between Python UDFs and Pandas UDFs? A: Both

require explicit handling — nulls typically appear as `None` in row UDFs and as `NaN`/`None` within the `pd.Series` for Pandas UDFs; failing to guard against them (e.g., calling a method on a `None`/`NaN` value) can raise runtime errors or produce incorrect results in either case.

Q12. [Output] How would you identify Python UDF bottlenecks in the Spark UI physical plan? A:

Look for `BatchEvalPython` (row UDFs) or `ArrowEvalPython` (Pandas UDFs) operators in the physical plan, and correlate with executor-side CPU/time attributed to spawned Python worker processes in the Executors tab.

Q13. [Internals] Why is Python UDF slower? A: Per-row serialization/deserialization across the JVM→Python process boundary, no participation in Catalyst-level plan optimization (pushdown/pruning around it), no whole-stage codegen fusion, and Python interpreter overhead per invocation — all compounding relative to a native expression.

Q14. [Internals] What is Arrow, precisely, and why does it enable fast JVM→Python exchange? A: A

cross-language, columnar, in-memory data format designed for zero/low-copy data sharing between processes/languages without per-row serialization — because both JVM and Python can interpret the same memory layout directly (or with minimal conversion), it avoids the expensive object-by-object (de)serialization that row-based transfer requires.

Q15. [Internals] Why are Pandas UDFs faster? A: Batch/vectorized processing via Arrow reduces the number of expensive cross-process round trips (per batch instead of per row) and leverages vectorized pandas/numpy operations internally, which are themselves far more CPU-efficient than Python's per-element interpreter loop.

Q16. [Internals] When would you use a Pandas UDF? A: For complex per-row or per-group logic that can't be expressed via native Spark SQL functions — e.g., custom statistical computations, calling into external

ML/scientific libraries (numpy/scipy/sklearn), or per-group time-series-style transformations — where vectorization can still be exploited within the batch.

Q17. [Internals] Why are built-in functions still preferred over Pandas UDFs even when equivalent logic exists natively? A: Native functions retain full Catalyst optimization

(pushdown/pruning/reordering) and whole-stage codegen eligibility with zero cross-process overhead at all — Pandas UDFs, while much better than row UDFs, still can't match that level of engine-native efficiency.

Q18. [Follow-up] If Pandas UDFs use Arrow, why not always use them instead of native functions by default? A: Native functions still outperform due to full Catalyst/codegen integration; Pandas UDFs

remain a fallback for expressiveness (logic not achievable natively), not a default choice to reach for when native alternatives exist.

Q19. [Follow-up] How would you tune `spark.sql.execution.arrow.maxRecordsPerBatch` and why does it matter? A: It balances per-batch memory footprint (larger batches reduce round-trip overhead but

increase peak memory per task) against the number of JVM→Python round trips (smaller batches mean more frequent, higher-overhead crossings) — tune upward for memory-light computations to reduce overhead, downward if executors show memory pressure from oversized batches.

Q20. [Follow-up] Explain Grouped Map / Cogrouped Pandas UDFs — when would you reach for them over scalar Pandas UDFs? A: Use them for genuine group-wise custom transformations that can't be

expressed as simple column-wise scalar operations — e.g., fitting a separate per-group time-series model, or combining/aligning two grouped datasets (cogroup) for custom joint processing — where the function needs the *entire* group's data (as a pandas DataFrame) at once rather than operating independently per row/batch.

Q21. [Scenario] Would converting a Python UDF to a Pandas UDF change the physical plan's join strategy elsewhere in the query? A: No — UDF-to-Pandas-UDF conversion affects only the execution

mechanics of that specific operation (`BatchEvalPython` → `ArrowEvalPython`); it doesn't change unrelated cost-based decisions like join strategy elsewhere in the plan, unless it happens to change estimated output size/statistics in a way that influences those separate decisions.

Q22. [Trap] Does using a Pandas UDF guarantee Catalyst can push a filter through it the way it could with a native function? A: No — Pandas UDFs remain opaque to Catalyst's pushdown/pruning logic just

like row UDFs; the vectorization improvement is purely about execution-transport efficiency, not about restoring plan-level optimization transparency.

Q23. [Coding] How would you profile whether a Pandas UDF's bottleneck is the Arrow transport or the actual function logic? A: Compare wall-clock time of an artificially trivial Pandas UDF (e.g., simple

pass-through) against the real UDF on the same data volume — if the trivial version is still slow, transport/overhead dominates; if the trivial version is fast and only the real logic is slow, the function's internal computation is the bottleneck, guiding whether to optimize the algorithm itself or the batching configuration.

Q24. [Scenario] Would increasing `maxRecordsPerBatch` always help a memory-bound Pandas UDF job? A:

No — increasing batch size for an already memory-constrained job would likely worsen OOM risk, since larger batches mean more data materialized in executor memory per task at once; for memory-bound cases, *decreasing* batch size (trading more overhead for lower peak memory) is often the appropriate direction instead.

Q25. [Internals] How does Arrow's columnar layout specifically benefit a Pandas UDF operating on a single numeric column, compared to row-based transfer of the same data? A: Columnar layout means

that single column's values are stored contiguously, enabling the pandas/numpy vectorized operation to process them as a tight, cache-friendly array operation, whereas row-based transfer would require reconstructing/extracting that column's values from interleaved per-row structures — the columnar alignment directly matches how vectorized numeric operations want to consume data.

Q26. [Follow-up] Would you recommend Pandas UDFs for streaming (Structured Streaming) pipelines the same way as batch? A: The same general Python/Pandas UDF vs native tradeoffs apply, but the added latency-sensitivity of streaming micro-batches means the relative cost of UDF overhead can be more noticeable per-trigger; still generally preferable to prefer native functions, using Pandas UDFs only where genuinely necessary, with attention to per-batch processing time staying within the trigger interval.

Q27. [Scenario] A teammate proposes rewriting an entire complex feature-engineering pipeline as one giant Pandas UDF "for simplicity." What's your pushback? A: While potentially simpler to write, this discards Catalyst's ability to optimize/reorder the individual steps and forces the entire computation through the JVM↔Python boundary as an opaque block — likely far slower and less maintainable at scale than expressing as much as possible via native operations, reserving a (possibly smaller, more targeted) Pandas UDF only for the specific logic that truly requires it.

Q28. [Trap] Is `spark.sql.execution.arrow.pyspark.enabled` required for Pandas UDFs to function at all, or just for performance? A: Arrow-based execution is fundamentally how Pandas UDFs operate (they're defined around Arrow-batched Series/DataFrame inputs/outputs) — it's not merely an optional performance toggle for this UDF type, though the general Arrow-enabled flag also affects other conversions like `toPandas()`.

Q29. [Coding] How would you handle a Pandas UDF needing access to a large read-only lookup table alongside its batch input? A: Use a broadcast variable to distribute the lookup table once to each executor (rather than shipping it repeatedly per task/batch), and reference the broadcasted value's `.value` inside the Pandas UDF function body for local, in-memory lookups during batch processing.

Q30. [Follow-up] What's your overall decision framework for choosing among native functions, Scala UDFs, Python UDFs, and Pandas UDFs? A: Default to native Spark SQL functions whenever functionally possible (best optimization/performance); if custom logic is required and the team has Scala/JVM capability, a Scala UDF avoids the cross-language serialization cost while accepting some lost Catalyst/codegen transparency; if Python-specific libraries/logic are required, prefer a Pandas UDF over a row UDF for the vectorization benefit, reserving plain row UDFs for the rare case where per-row logic can't be vectorized at all.

Module 13: Memory Management

Q1. [Scenario] `groupBy` fine at 100GB, spills heavily at 500GB on the same cluster — walk through remediation. A: The aggregation's per-task memory requirement scales with data volume per partition; at 500GB, the same shuffle-partition count now yields much larger per-task buffers exceeding available execution memory, forcing spill. Remediation: increase `spark.sql.shuffle.partitions` to shrink per-task data, add executor memory/executors, check for/address any skew concentrating extra load on specific tasks, and consider pre-filtering to reduce data volume before the aggregation.

Q2. [Scenario] Executor OOMs during a large shuffle-heavy join; driver memory looks fine — where did it fail? A: Likely within execution memory during the join's shuffle/sort or hash-table-building buffers on the executor — building a shuffle hash table or handling sort-merge join buffers for an

oversized/skewed partition can exceed available execution memory, independent of driver-side memory which is unaffected by executor-side task processing.

Q3. [Scenario] Increasing `spark.executor.memory` doesn't fix the same OOM point — what else might be the bottleneck? A: Data skew concentrating disproportionate data onto one task (no amount of *uniform* extra memory across all executors helps a single overloaded task beyond a point), too few partitions concentrating data per task, off-heap/overhead memory misconfiguration, or the driver-side broadcast size being the actual culprit rather than executor task memory.

Q4. [Scenario] Explain the unified memory model's dynamic sharing between execution and storage memory. A: Execution and storage memory draw from a single unified pool; under memory pressure, execution memory can evict cached (storage) blocks to reclaim space for active computation, since execution has priority, while storage cannot forcibly evict execution memory currently in active use — subject to a configurable minimum storage guarantee (`spark.memory.storageFraction`) protecting some cached data from being fully evicted.

Q5. [Trap] Does increasing executor memory always fix OOM errors? A: Not necessarily — if the root cause is data skew (a single task disproportionately overloaded), adding memory uniformly across all executors doesn't specifically address that one task's excess load; the skew itself needs to be fixed (salting, AQE skew handling, repartitioning).

Q6. [Trap] Do cached data and shuffle/aggregation buffers compete for the exact same memory pool? A: True under Spark's unified memory model — both draw from the same unified region, dynamically balanced between storage and execution use based on demand and configured fractions.

Q7. [Trap] Does driver OOM only happen from `.collect()`? A: No — also from large broadcast joins (small table collected to driver before broadcasting), accumulators holding excessive state, or building/planning extremely complex query plans (many unions/joins) that consume driver memory during planning itself.

Q8. [Trap] Is spill to disk always a bug/misconfiguration? A: Not necessarily — occasional modest spill is a normal safety mechanism preventing OOM; *heavy, consistent* spill across many tasks indicates a real tuning problem worth addressing (undersized partitions/memory relative to data volume).

Q9. [Coding] How would you use Spark UI's Executors tab to distinguish execution-memory-driven OOM from storage-memory-driven OOM? A: Check the "Storage Memory" used/available column alongside task-level failure details — if cached data usage is high and tasks fail during a wide computation (join/aggregation) rather than during a cache operation, execution memory pressure (competing with storage) is more likely the cause; persistent high storage usage crowding out execution room can be confirmed by comparing before/after `unpersist()` on non-essential cached DataFrames.

Q10. [Coding] Edge case: caching a DataFrame larger than total cluster memory with `MEMORY_ONLY` — what happens to the excess? A: The fraction that doesn't fit is simply dropped/not cached (not stored anywhere) under `MEMORY_ONLY`; those specific partitions are recomputed on demand via lineage whenever referenced again, potentially causing repeated, expensive recomputation for that uncached portion — `MEMORY_AND_DISK` avoids this by spilling the excess to disk instead of dropping it.

Q11. [Output] Which Spark UI Stages tab columns indicate memory pressure most directly? A: "Spill (Memory)" and "Spill (Disk)" columns — non-zero "Spill (Disk)" specifically confirms that memory-resident intermediate data exceeded available execution memory and had to be written to disk.

Q12. [Internals] Explain execution, storage, user, and reserved memory within an executor's heap. A: Reserved memory: a small fixed portion set aside for Spark's internal bookkeeping, not user-configurable in practice. User memory: holds user data structures and metadata not managed by Spark's memory

manager (e.g., objects created inside UDFs). Unified memory (execution + storage): the large dynamically-shared pool used for shuffle/join/aggregation buffers (execution) and cached/persisted data (storage), governed by `spark.memory.fraction` and `spark.memory.storageFraction`.

Q13. [Internals] Why does `groupBy()` sometimes spill to disk? A: Because the aggregation/shuffle requires holding intermediate hash-table or sort buffers that exceed available execution memory for that task, causing Spark to spill intermediate data to disk to avoid an outright OOM failure.

Q14. [Internals] How do `spark.memory.fraction` and `spark.memory.storageFraction` influence the unified pool split? A: `spark.memory.fraction` sets what portion of total heap (after reserved memory) is available to the unified execution+storage pool at all (vs "user memory"); `spark.memory.storageFraction` sets the minimum guaranteed portion of that unified pool reserved for storage (cached data) that execution memory cannot forcibly evict below.

Q15. [Internals] Practical impact of enabling off-heap memory on GC behavior and OOM risk? A: Off-heap allocation for Tungsten-managed memory reduces JVM garbage collector involvement and pause times for that memory region (since it's outside GC-managed heap), potentially improving throughput/latency consistency; however it requires careful sizing (`spark.memory.offHeap.size`) since off-heap OOM manifests differently (native memory allocation failures) than typical heap OOM.

Q16. [Follow-up] Without adding cluster resources, top 3 levers to fix an OOM/spill problem? A: (1) Increase shuffle partitions to shrink per-task data volume, (2) diagnose and fix skew via salting or AQE skew join handling, (3) filter/select earlier in the pipeline to reduce data volume reaching the memory-intensive operation, and reduce unnecessary caching competing for the same memory pool.

Q17. [Follow-up] How does memory pressure differ between a wide join OOM and a groupBy OOM in root cause? A: Join OOM often stems from building an in-memory hash table (broadcast join collecting/broadcasting a mis-sized "small" table, or shuffle hash join building per-partition hash tables) versus groupBy OOM stemming from sort/hash aggregation buffers during grouping — both are ultimately execution-memory pressure, but the specific operator-level mechanism causing it differs.

Q18. [Follow-up] How does AQE's skew handling reduce OOM risk without adding cluster resources? A: By splitting an oversized skewed partition into multiple smaller sub-tasks processed independently (each handling a fraction of the original overloaded partition's data), AQE directly reduces the peak memory required per task, avoiding the single-task OOM that an unmitigated skewed partition would otherwise cause.

Q19. [Scenario] A cached DataFrame occupies most of storage memory, and a later `groupBy` on unrelated data starts spilling heavily. Related? A: Potentially yes — if the cached data is consuming a large share of the unified memory pool and execution memory can't reclaim enough (bounded by `storageFraction`'s guaranteed minimum for storage), the `groupBy`'s execution memory needs may be artificially constrained; unpersisting unnecessary cached data or adjusting `storageFraction` could free up execution headroom.

Q20. [Trap] Does "unified memory management" mean execution and storage always split 50/50? A: No — it's a *dynamic* split based on demand, not a fixed 50/50; `storageFraction` sets a protected minimum for storage, but the actual real-time allocation between execution and storage shifts based on which is under active demand at any given moment, up to the boundaries configured.

Q21. [Coding] How would you programmatically monitor memory spill trends across many job runs for regression detection? A: Pull stage-level metrics (`spillDiskBytes`, `spillMemoryBytes` or equivalent fields) via the Spark REST API or parsed event logs for each run, store them in a tracking table/dashboard over

time, and alert when spill volume trends upward significantly relative to historical baselines for the same job.

Q22. [Scenario] Would caching an intermediate DataFrame ever *reduce* OOM risk for a downstream memory-intensive operation? A: Potentially yes, if caching avoids expensive, memory-heavy recomputation of an upstream chain multiple times (each recomputation itself risking its own memory pressure); but if the cache itself competes significantly with the downstream operation's execution memory needs, it could also worsen things — the net effect depends on the specific memory budget and reuse pattern, requiring case-by-case evaluation.

Q23. [Internals] How does `spark.executor.memoryOverhead` factor into avoiding OOM beyond `spark.executor.memory` itself? A: It reserves additional memory (outside the JVM heap governed by `spark.executor.memory`) for JVM overhead, native libraries, and (critically for PySpark) spawned Python worker process memory — insufficient overhead sizing can cause the cluster manager (YARN/K8s) to kill the executor container for exceeding physical memory limits, even if the JVM heap itself reports no OOM.

Q24. [Follow-up] Would a job with heavy Python UDF usage need a different memory tuning approach than a pure DataFrame job? A: Yes — beyond standard execution/storage memory tuning, you'd specifically need to size `spark.executor.memoryOverhead` (or `spark.executor.pyspark.memory` where available) generously enough to accommodate the memory consumed by spawned Python worker processes, which sits outside the JVM heap Spark's unified memory model directly manages.

Q25. [Scenario] Two jobs on the same cluster, one caches a huge DataFrame and leaves it cached indefinitely, the other later OOMs during a join. Root cause and fix? A: The abandoned cached DataFrame from the first job is consuming shared cluster memory (competing with the second job's execution memory needs, especially in a shared/multi-tenant cluster context); fix: explicitly `unpersist()` DataFrames once no longer needed, and consider memory isolation/quota policies for shared clusters running multiple concurrent workloads.

Q26. [Trap] Does a successful job (no failures, no OOM) guarantee its memory configuration is well-tuned? A: No — a job can succeed while still spilling heavily or running with excessive memory headroom (over-provisioned); "no failure" only confirms memory was *sufficient*, not that it was efficiently or cost-effectively sized.

Q27. [Coding] How would you deliberately reproduce and observe a controlled OOM/spill scenario in a test environment to build intuition? A: Run a `groupBy`/join on a moderately large synthetic dataset with a deliberately low `spark.sql.shuffle.partitions` and constrained executor memory, then observe Spark UI's spill and GC metrics as you incrementally adjust partition count/memory settings, directly correlating configuration changes with memory behavior.

Q28. [Internals] How does garbage collection interact with execution memory pressure specifically (not just general JVM overhead)? A: Heavy execution memory churn (e.g., from RDD/UDF-heavy code generating many short-lived objects, or frequent spill-related buffer allocation/deallocation) increases GC frequency/pause time, which itself consumes CPU cycles that could otherwise go toward actual computation — compounding the performance cost of memory pressure beyond just the direct spill I/O cost.

Q29. [Scenario] Would switching from `MEMORY_ONLY` to `MEMORY_AND_DISK` caching ever cause a *previously working job to newly OOM*? A: Unlikely to directly cause new OOM (it's generally a safer option, spilling excess to disk instead of dropping/recomputing), though it can change performance characteristics (disk I/O for spilled cache portions) — if anything, it should reduce OOM risk relative to `MEMORY_ONLY`'s silent-drop behavior, not increase it.

Q30. [Follow-up] What's your systematic checklist when facing an unfamiliar OOM/spill issue in production? A: (1) Identify exactly where in the plan/stage the failure occurs via Spark UI/stack trace, (2) check for skew via task duration/shuffle-read distribution, (3) review shuffle partition count relative to data volume, (4) check for excessive/unnecessary caching competing for memory, (5) verify `memoryOverhead` sizing especially for UDF-heavy jobs, (6) only after ruling out the above, consider scaling up executor memory/count as a resource-level fix.

Module 14: Cache and Persist

Q1. [Scenario] Cache a DataFrame, run one action, then modify upstream lineage (new source read/filter) — does cached data reflect the new lineage? A: No — the cache is tied to the specific DataFrame/lineage object at the time `.cache()` was called; a newly constructed DataFrame with different upstream lineage is an entirely separate object and won't automatically reuse or be affected by the old cached data.

Q2. [Scenario] Caching 5 large DataFrames "just in case" causes slowdowns/spilling — why did caching hurt? A: The cached data competes with execution memory in the unified memory pool; with so much memory committed to caching, execution memory available for actual computation (joins/aggregations) shrinks, forcing more spill/eviction than if less had been cached — over-caching becomes a net negative.

Q3. [Scenario] `.cache()` called but no action runs before writing downstream — was the cache useful? A: No — caching only provides benefit when the same materialized data is reused across *multiple* actions; if the cached DataFrame is consumed only once downstream (directly feeding the write), there was no repeated computation to save, so the caching overhead (materializing and storing it) provided no benefit.

Q4. [Scenario] Real pipeline scenario where `persist(MEMORY_AND_DISK)` is clearly better than plain `.cache()`? A: Note first: DataFrame `.cache()` in modern Spark already defaults to `MEMORY_AND_DISK` storage level (unlike RDD's `MEMORY_ONLY` default) — a common trap to clarify. For genuinely large intermediate results reused across multiple downstream branches (e.g., a heavily-filtered/joined DataFrame feeding three separate aggregation queries) where the data may not fully fit in memory, `MEMORY_AND_DISK`-style persistence ensures the excess spills reliably to disk rather than being dropped and recomputed.

Q5. [Trap] Does calling `cache()` immediately execute the DataFrame? A: No — caching is lazy; an action materializes the cached data on its first execution after `.cache()` is called.

Q6. [Trap] Are `.cache()` and `.persist()` functionally different methods? A: Not fundamentally — `.cache()` is shorthand for `.persist()` using a default storage level; `.persist(storageLevel)` allows explicitly specifying a different storage level.

Q7. [Trap] Does uncached data get recomputed from scratch every time it's referenced in a new action? A: Yes, unless cached/persisted or checkpointed — each new action recomputes the full lineage from source by default.

Q8. [Trap] Is caching always beneficial for iterative algorithms? A: Generally yes for genuinely reused data across iterations, but can hurt if the data doesn't fit in available memory, causing eviction/spill overhead that may exceed the cost of simply recomputing from a shorter lineage.

Q9. [Coding] Given a cached `df` and a filtered `df2` derived from it, is `df2` served from cache or recomputed from source? A: Served from `df`'s cached data — since `df2`'s lineage derives from the already-cached `df`, subsequent actions on `df2` reuse the cached materialized data as their starting point rather than re-reading/reprocessing from the original source.

Q10. [Coding] Edge case: calling `.unpersist()` while a job is actively reading from the cached data? A: Behavior depends on the `blocking` parameter — with blocking, it waits until safe to remove; without blocking (or depending on timing), an in-flight job might end up recomputing the affected data from source if it's evicted mid-use, a subtle race condition worth being aware of in concurrent/multi-job environments.

Q11. [Output] What would Spark UI's Storage tab show for a DataFrame cached with `MEMORY_AND_DISK` that partially spilled? A: A breakdown showing some fraction of the cached size as "In-memory" and the remaining fraction as "On Disk," reflecting the portion that didn't fit in available execution/storage memory and was spilled.

Q12. [Internals] List storage levels and when each is appropriate. A: `MEMORY_ONLY` (fastest, but drops data that doesn't fit — risk of recomputation); `MEMORY_AND_DISK` (spills excess to disk instead of dropping — safer default for large data); `MEMORY_ONLY_SER`/`MEMORY_AND_DISK_SER` (serialized storage, more compact but costs CPU to deserialize on access — useful when memory is tight); `DISK_ONLY` (fully disk-backed, slowest but most memory-conservative); replicated variants (e.g., `MEMORY_ONLY_2`) add fault tolerance by storing extra copies across executors, at higher storage cost.

Q13. [Internals] Concrete case where caching helps. A: An iterative ML training loop reusing the same feature DataFrame across many training iterations — without caching, each iteration would recompute the entire upstream feature-engineering pipeline from source, multiplying cost by the iteration count.

Q14. [Internals] Concrete case where caching hurts. A: A DataFrame used exactly once downstream (single-use, no reuse) cached "defensively" — the caching overhead (materialization + memory pressure on other concurrent operations) provides no offsetting benefit since there's no repeated computation being saved.

Q15. [Internals] How does cache eviction (LRU) interact with active jobs needing that data? A: If memory pressure forces eviction of a cached partition currently needed by an active or upcoming job, Spark transparently recomputes that specific partition from its lineage when next accessed — correctness is preserved, but at the cost of the recomputation Spark was trying to avoid via caching in the first place.

Q16. [Follow-up] How is `.cache()` different from `.checkpoint()` regarding lineage and fault tolerance? A: Cache preserves the original lineage (allowing recomputation if cached data is lost/evicted); checkpoint truncates lineage entirely, writing data reliably to durable storage such that recovery relies on the durably-stored checkpoint data itself, not on replaying the original (now-discarded) transformation chain.

Q17. [Follow-up] If a filter is applied before every use of a DataFrame, should you cache before or after the filter? A: After the filter — cache only the data that's actually reused downstream, avoiding wasted memory caching rows that get filtered out on every subsequent use anyway.

Q18. [Follow-up] How do you decide whether a pipeline stage should be cached, checkpointed, or just recomputed? A: Consider reuse frequency (multiple actions benefit from caching; single-use doesn't), cost of recomputation (expensive upstream joins/reads favor caching or checkpointing), lineage length/fault-tolerance needs (very long lineages favor checkpointing to bound recovery cost), and available memory (insufficient memory for the data favors checkpointing to durable storage over risky/partial caching).

Q19. [Follow-up] Risk of forgetting `.unpersist()` on long-lived cached DataFrames in a shared interactive cluster? A: Gradual memory exhaustion across many users/sessions sharing the same cluster, as abandoned cached data accumulates and is never reclaimed until the cluster/session restarts — a common real-world resource-hygiene issue in shared notebook environments.

Q20. [Scenario] Would caching a DataFrame that's about to be written to disk (single downstream consumer: the write) ever still make sense? A: Generally no, per Q3's logic — unless the write itself triggers multiple internal passes over the data (rare, but possible for certain output formats/partitioning strategies) or you also plan additional actions on the same DataFrame in the same session beyond that single write, caching purely for a single write provides no reuse benefit.

Q21. [Trap] Does `.cache()` guarantee data will be read from memory on the next access, even under memory pressure? A: No — under the default `MEMORY_AND_DISK`-style behavior (or explicit disk-inclusive storage levels), data may be spilled to disk under pressure and read from there instead of memory; only `MEMORY_ONLY` risks outright dropping (not spilling) data that doesn't fit, with no disk fallback.

Q22. [Coding] How would you verify programmatically whether a specific DataFrame is currently cached? A: `df.storageLevel` (or `df.is_cached` in some API surfaces) can be inspected to check the current caching status and storage level configuration for that DataFrame.

Q23. [Scenario] Would caching help if the bottleneck is actually disk I/O on the initial source read, not repeated computation? A: Yes, indirectly — if the source read itself is expensive (e.g., slow network storage) and the same read result is reused across multiple actions, caching avoids repeating that expensive read multiple times, even though the "savings" here are about I/O rather than CPU-bound recomputation specifically.

Q24. [Trap] Is checkpointing always slower than caching due to writing to durable storage? A: Generally checkpointing has higher upfront cost (durable write to distributed storage vs in-memory/local-disk caching), but this cost is offset by eliminating the risk of expensive full-lineage recomputation on failure for very long/complex pipelines — the "slower" upfront cost buys more reliable, bounded-cost recovery.

Q25. [Coding] How would you benchmark whether caching actually helped a specific multi-action pipeline? A: Run the pipeline with and without `.cache()` (same data/cluster), measuring total wall-clock time across all actions and Spark UI stage counts/duration for each variant — a meaningful reduction in total duration and skipped/reused stages in the cached version confirms the benefit; a negligible or negative difference suggests caching wasn't warranted for that specific reuse pattern.

Q26. [Scenario] Would you ever explicitly choose `DISK_ONLY` caching over just not caching at all? A: Yes — for a DataFrame that's expensive to recompute (e.g., involves a costly external API call inside a UDF, or an extremely deep/expensive lineage) but too large to fit in available memory, `DISK_ONLY` still avoids full recomputation on reuse (reading from disk instead), which can be much cheaper than recomputing from scratch even though it's slower than an in-memory cache would be.

Q27. [Follow-up] How would you handle a scenario where multiple concurrent jobs on a shared cluster all want to cache large, overlapping datasets? A: Consider consolidating overlapping caching needs into a single shared cached DataFrame (if the jobs can share the same upstream computation), apply cluster-level memory governance/quotas to prevent any single job from monopolizing cache memory, and proactively `unpersist()` once each job's need for that cached data has passed.

Q28. [Scenario] A cached DataFrame's underlying source data changes in place (e.g., source files overwritten) after caching. Does the cache reflect the new data? A: No — the cached data is a materialized snapshot from when it was computed; it does not automatically refresh to reflect changes to the underlying source made afterward, potentially leading to stale-data bugs if not `unpersist()`-ed and recomputed deliberately after the source change.

Q29. [Trap] Does `.cache()` on a DataFrame automatically cache any derived DataFrames created from it afterward? A: No — caching applies specifically to the DataFrame it's called on; derived DataFrames

reuse the *cached data* as their computation starting point (avoiding recomputation of the shared upstream lineage), but they themselves aren't automatically cached unless you explicitly call `.cache()` on them too.

Q30. [Follow-up] What's your production checklist for using caching responsibly in a shared, long-running cluster environment? A: Cache only genuinely reused DataFrames (verified reuse across multiple actions), cache as late as possible in the pipeline (after filtering to minimize cached volume), explicitly `unpersist()` once no longer needed, monitor Storage tab usage over time to catch abandoned caches, and prefer checkpointing over caching for very long lineages needing reliable fault tolerance rather than just performance reuse.